

MediBook Phase 3

Slot Management System

COMPLETE LEARNING GUIDE

Time & Date Algorithms · Concurrency Control
Per-Day Schedules · Holiday Auto-Cancel · Optimistic Locking

What You Built in Phase 3

DoctorSlotStatus enum (4-state lifecycle)

4 entities with complex relationships

4 repositories with derived queries + JPQL + Specs

11 DTOs for clean API contracts

3 mappers with bidirectional collection handling

Slot Generation Algorithm with $O(1)$ holiday lookups

3 services orchestrating slot operations

3 controllers (Doctor, Admin, Public)

Holiday auto-cancel feature (your design choice!)

@Version + @Lock for concurrency control

14 endpoints tested end-to-end

Smart indexing strategy for scale

Stack: Java 21 · Spring Boot 3.5 · MySQL 8 · JPA · Spring Security

Repository: github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook

Table of Contents

PART I DESIGN & ARCHITECTURE

1. Business Requirements
2. Why 4 Entities (Normalization)
3. Slot Lifecycle State Machine
4. Complete LLD Diagram
5. API Design (14 Endpoints)

PART II DATA LAYER

6. DoctorSlotStatus Enum
7. DoctorSlotConfig Entity (Master Config)
8. DoctorWeeklySchedule Entity (Per-Day Hours)
9. DoctorHoliday Entity
10. DoctorSlot Entity (THE BIG ONE)
11. Indexing Strategy Deep Dive
12. Optimistic vs Pessimistic Locking

PART III REPOSITORIES & QUERIES

13. The 4 Repositories
14. Derived Queries, JPQL, and Specifications
15. Bulk Operations with @Modifying
16. Composite Index Left-Prefix Rule

PART IV DTO & MAPPING

17. 11 DTOs and Their Purposes
18. Nested DTOs with @Valid
19. Bidirectional Collection Management

PART V SLOT GENERATION ALGORITHM

20. The Algorithm Explained
21. Performance Optimizations (Set, Map)
22. Date Iteration with LocalDate
23. Worked Example: 88 Slots in 7 Days

PART VI SERVICE LAYER

24. DoctorSlotConfigService
25. DoctorHolidayService with Auto-Cancel
26. DoctorSlotService
27. Defense in Depth Validation

PART VII CONTROLLERS & SECURITY

28. Three Controllers Pattern
29. @DateTimeFormat for Date Parsing
30. Regex in URL Patterns (Security!)
31. SecurityConfig Updates

PART VIII REFERENCE

32. All Endpoints Cheat Sheet
33. Postman Test Suite Recap
34. Common Mistakes & Fixes
35. Interview Q&A; for Phase 3

36. Phase 4 Preview: Booking System

PART I · DESIGN & ARCHITECTURE

1. Business Requirements

Phase 3 builds the foundation for appointment booking by creating a slot management system. It must support real-world hospital scheduling complexity while remaining performant for thousands of slots across hundreds of doctors.

Functional Requirements

- Admin configures each doctor's working pattern (per-day hours, slot duration)
- Different working hours per day (Mon-Fri full, Sat half, Sun closed)
- Generate hundreds of slots from configuration + date range
- Skip holidays during generation
- Auto-cancel existing slots when holiday is added later
- Public browsing of AVAILABLE slots only (filter hidden statuses)
- Doctor and Admin can cancel individual slots
- Audit trail for all cancellations

Non-Functional Requirements

- **Performance:** Handle 60K+ slots with sub-second queries
- **Concurrency:** Prevent double-booking via optimistic locking (Phase 4 prep)
- **Security:** Role-based access (DOCTOR/ADMIN/PUBLIC)
- **DOS Protection:** Cap date ranges in public endpoints
- **Data Integrity:** Unique constraints prevent duplicate slots

INTERVIEW POINT » Interview Q: "Walk me through a complex feature you've designed."

A: "I designed a slot management system with per-day working schedules, holiday handling, and intelligent slot generation. The key challenge was modeling real hospital complexity: doctors work different hours each day, take holidays, and slots must be pre-generated for booking. I split the schema into four entities for normalization, implemented an $O(n)$ generation algorithm with $O(1)$ lookups, and added optimistic locking via @Version for future booking concurrency. End-to-end tested with Postman, all flows validated."

2. Why 4 Entities (Normalization)

One Table vs Four Tables

ALTERNATIVE: Cram everything into ONE table

```
doctor_slot_settings {  
  doctor_id,  
  monday_start, monday_end,  
  tuesday_start, tuesday_end,  
  ...sunday_start, sunday_end,  
  slot_duration,  
  max_bookings,  
  holidays_json,  
  slots_json  
}
```

PROBLEMS:

- 14 columns just for working hours (mon-sun start/end)
- JSON columns can't be indexed efficiently
- Querying "doctors working Saturday" = JSON parsing
- Adding a holiday = rewriting entire JSON column
- Slot count grows huge (1000s) - won't fit in JSON
- Bulk operations impossible

OUR DESIGN: Four focused entities

```
DoctorSlotConfig      (1 row per doctor)  
+ DoctorWeeklySchedule (7 rows per config)  
+ DoctorHoliday       (many rows per doctor)  
+ DoctorSlot          (thousands of rows per doctor)
```

BENEFITS:

- Each entity has one concern (Single Responsibility)
- Index any column for fast queries
- Bulk operations work (UPDATE many slots at once)
- Atomic changes (add holiday without touching slots config)
- Scales to millions of rows

The Relationships

Entity Graph

DOCTOR (Phase 2)

| 1:1
v

DoctorSlotConfig (master config)

| 1:7
v

DoctorWeeklySchedule (one per day MON-SUN)

DOCTOR

| 1:N
v

DoctorHoliday (many holidays per doctor)

DOCTOR

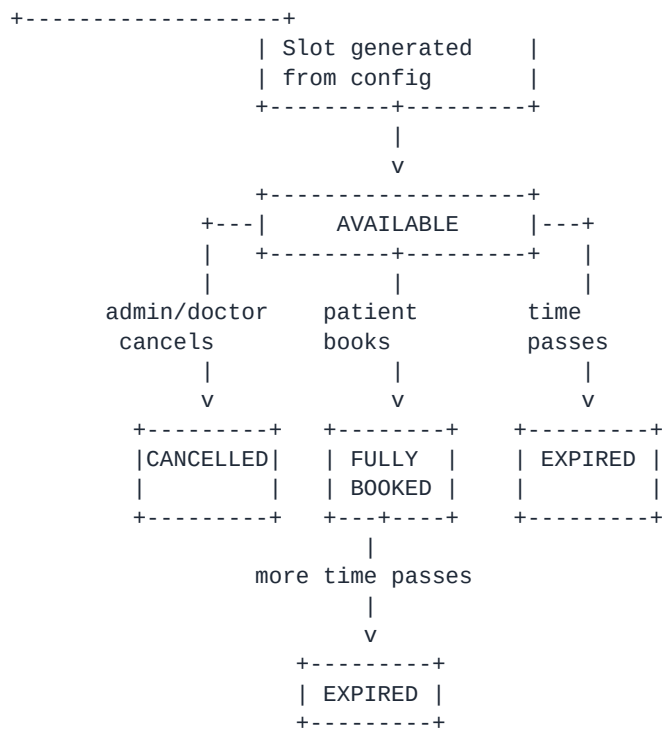
| 1:N
v

DoctorSlot (thousands of slots per doctor)

Why this matters » Normalization isn't just academic — each query pattern in our system maps to ONE entity. Patient browsing = DoctorSlot only. Schedule changes = DoctorWeeklySchedule only. Holiday management = DoctorHoliday only. Separation makes the system simple.

3. Slot Lifecycle State Machine

Slot State Machine



ENFORCED IN SERVICE:

cancelSlot() throws if status == CANCELLED (already cancelled)
cancelSlot() throws if status == EXPIRED (too late)
bookSlot() only allowed if status == AVAILABLE (Phase 4)

Why Track States Explicitly?

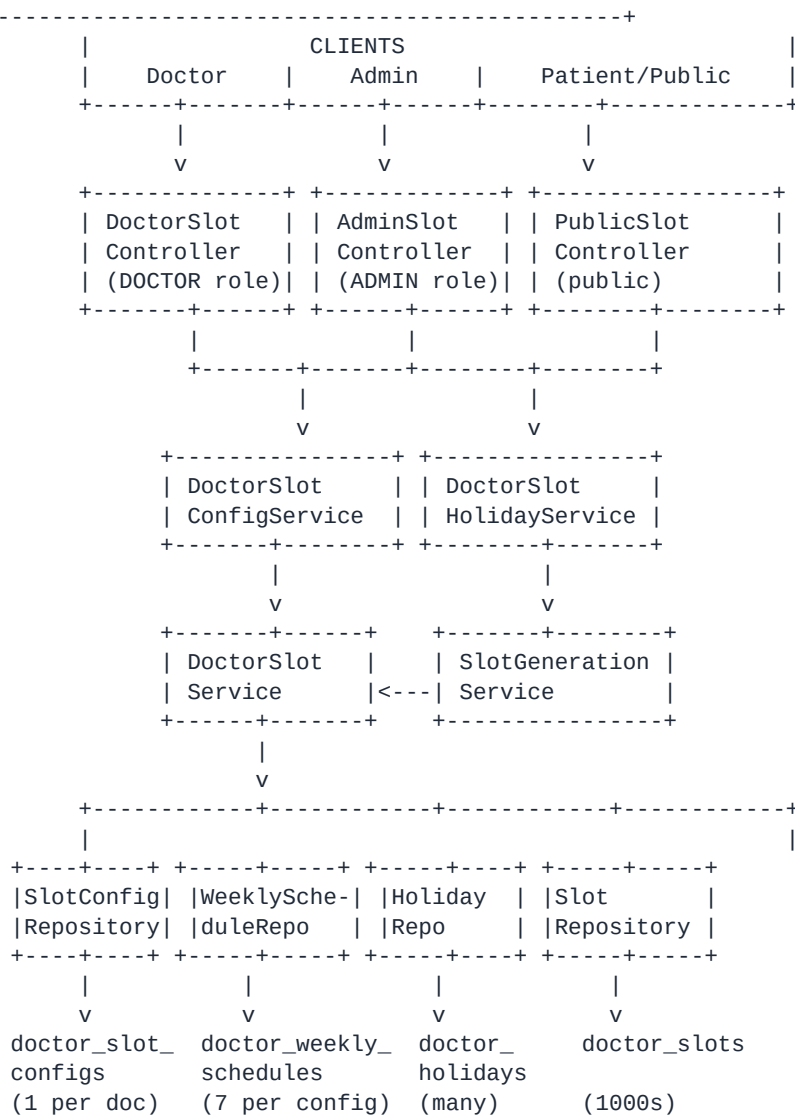
Alternative: calculate status from `current\_bookings >= max\_bookings`. But this can't represent CANCELLED or EXPIRED - those need explicit tracking. Storing the status enables all four lifecycle states to be queried, filtered, and reported uniformly.

INTERVIEW POINT » Interview Q: "How do you handle state transitions in your system?"

A: "State machines in the service layer with explicit validation. Before any transition, I check the current state. For example, before cancelling a slot, I verify it's not already CANCELLED or EXPIRED. Invalid transitions throw `BadRequestException`. This centralized validation prevents inconsistent state and is independent of which controller or API endpoint calls it."

4. Complete LLD Diagram

Phase 3 LLD



5. API Design (14 Endpoints)

Doctor Endpoints (DOCTOR role)

Method	Endpoint	Purpose
GET	/doctors/me/slots	View own slots paginated
GET	/doctors/me/slots/{id}	View specific slot detail
POST	/doctors/me/slots/{id}/cancel	Cancel own slot

Admin Endpoints (ADMIN role)

Method	Endpoint	Purpose
POST	/admin/doctors/{id}/slot-config	Create/update config
GET	/admin/doctors/{id}/slot-config	View config
PUT	/admin/doctors/{id}/slot-config/active	Toggle active
POST	/admin/doctors/{id}/slots/generate	Trigger generation
POST	/admin/doctors/{id}/holidays	Add single holiday
POST	/admin/doctors/{id}/holidays/bulk	Add bulk holidays
GET	/admin/doctors/{id}/holidays	List holidays
GET	/admin/doctors/{id}/holidays/upcoming	Upcoming holidays
DELETE	/admin/holidays/{id}	Remove holiday
GET	/admin/doctors/{id}/slots	View any doctor's slots
POST	/admin/slots/{id}/cancel	Cancel any slot

Public Endpoints (no auth)

Method	Endpoint	Purpose
GET	/doctors/{id}/slots/available	Available on date
GET	/doctors/{id}/slots/available/range	Available in range
GET	/doctors/{id}/slots/available/next	Next N available
GET	/doctors/{id}/slots/dates	Distinct dates

PART II · DATA LAYER

6. DoctorSlotStatus Enum

DoctorSlotStatus.java

```
public enum DoctorSlotStatus {  
    AVAILABLE,      // Has space, can be booked  
    FULLY_BOOKED,  // Reached max capacity  
    CANCELLED,     // Admin/doctor cancelled  
    EXPIRED        // Date/time has passed  
}
```

Why 4 States, Not More?

YAGNI (You Aren't Gonna Need It). Each state represents a distinct business condition. Adding LOCKED, PENDING, or RESERVED before having a real use case adds complexity without value. Phase 4 (Booking) will introduce these only if needed.

TIP » Always use `@Enumerated(EnumType.STRING)` to store as readable text. NEVER `ORDINAL` — reordering enum values silently corrupts existing data.

7. DoctorSlotConfig Entity (Master Config)

Schema

Master config schema

```
+-----+
|          DOCTOR_SLOT_CONFIGS          |
+-----+
| id                BIGINT  PK AUTO      |
| doctor_id         BIGINT  FK UNIQUE NOT NULL |
| slot_duration_minutes INT    NOT NULL DEFAULT 30 |
| max_bookings_per_slot INT    NOT NULL DEFAULT 2 |
| timezone          VARCHAR(50) NOT NULL |
| active            BOOLEAN NOT NULL DEFAULT 1 |
| created_at, updated_at  TIMESTAMP      |
+-----+
Has 7 children: DoctorWeeklySchedule (one per day)
```

Key Annotations

Critical relationship annotations

```
@OneToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "doctor_id", unique = true, nullable = false)
private Doctor doctor;

@OneToMany(mappedBy = "slotConfig",
            cascade = CascadeType.ALL,
            orphanRemoval = true,
            fetch = FetchType.LAZY)
private List<DoctorWeeklySchedule> weeklySchedules = new ArrayList<>();
```

Why cascade + orphanRemoval?

Cascade semantics

```
cascade = CascadeType.ALL:
  Save parent -> save children automatically
  Delete parent -> delete children
  No need to manually save each WeeklySchedule
```

```
orphanRemoval = true:
  Remove child from list -> JPA deletes from DB
  Critical for "rebuild schedule" updates:
    config.getWeeklySchedules().clear(); // DELETE old rows
    addWeeklySchedule(newOne);           // INSERT new rows
```

COMMON MISTAKE » Common mistake: Using `config.setWeeklySchedules(newList)` breaks `orphanRemoval`! JPA only tracks modifications to the ORIGINAL list instance. Setting a new list loses the orphan removal feature. Always `clear()` and `add()` on the existing list.

8. DoctorWeeklySchedule Entity (Per-Day Hours)

Why Separate Entity Instead of JSON?

Approach	Pros	Cons
JSON column	Quick to implement	No indexing, no SQL queries, fragile parsing
7 columns in config	Simple	14 nullable columns (mon_start, mon_end, ...)
Separate entity (chosen)	Indexed, queryable, normalized	More tables (acceptable trade)

Schema

Per-day schedule

```
+-----+
|          DOCTOR_WEEKLY_SCHEDULES          |
+-----+
| id          BIGINT  PK AUTO                |
| slot_config_id BIGINT  FK NOT NULL          |
| day_of_week  VARCHAR(10) NOT NULL          | enum string
| is_working_day BOOLEAN NOT NULL            |
| work_start_time TIME NULLABLE              | "10:00:00"
| work_end_time  TIME NULLABLE              | "20:00:00"
|
| UNIQUE (slot_config_id, day_of_week)       |
+-----+
```

Using Java's Built-in DayOfWeek

Using Java's DayOfWeek

```
import java.time.DayOfWeek;

@Enumerated(EnumType.STRING)
@Column(name = "day_of_week", nullable = false, length = 10)
private DayOfWeek dayOfWeek; // MONDAY, TUESDAY, ..., SUNDAY

// Convenience factory method:
public static DoctorWeeklySchedule nonWorkingDay(DayOfWeek dayOfWeek) {
    DoctorWeeklySchedule schedule = new DoctorWeeklySchedule();
    schedule.dayOfWeek = dayOfWeek;
    schedule.isWorkingDay = false;
    return schedule;
}
```

Why this matters » Java's `java.time.DayOfWeek` integrates with `LocalDate.getDayOfWeek()`. Creating our own enum would duplicate the standard library and lose the integration. Always prefer built-ins for well-known concepts.

9. DoctorHoliday Entity

Schema

Holiday schema

```
+-----+
|                DOCTOR_HOLIDAYS                |
+-----+
| id          BIGINT  PK AUTO                    |
| doctor_id   BIGINT  FK NOT NULL                |
| holiday_date DATE    NOT NULL                  |
| reason      VARCHAR(200) NOT NULL              |
| created_by_user_id BIGINT NULLABLE             |
| created_at   TIMESTAMP                          |
|                                                    |
| UNIQUE (doctor_id, holiday_date)               |
| INDEX  (doctor_id, holiday_date) - for range queries |
+-----+
```

Why LocalDate Not LocalDateTime?

Holidays are dates, not moments. Christmas is the whole day of Dec 25, not 'Dec 25 at 14:30'. Using LocalDate:

- Cleaner DB column (DATE vs TIMESTAMP)
- Date comparisons are unambiguous
- No timezone confusion (a date is the same date everywhere)

The composite index

Composite index

```
@Index(name = "idx_holiday_doctor_date",
      columnList = "doctor_id, holiday_date")
```

This index optimizes the slot generation algorithm's holiday lookup: **WHERE doctor_id = ? AND holiday_date BETWEEN ? AND ?**. Without it, generating 30 days of slots requires scanning the entire holidays table 30 times.

10. DoctorSlot Entity (THE BIG ONE)

Schema

Slot schema with indexes

DOCTOR_SLOTS			
id	BIGINT	PK AUTO	
doctor_id	BIGINT	FK NOT NULL	
slot_date	DATE	NOT NULL	
start_time	TIME	NOT NULL	
end_time	TIME	NOT NULL	
max_bookings	INT	NOT NULL DEFAULT 2	
current_bookings	INT	NOT NULL DEFAULT 0	
status	VARCHAR(20)	NOT NULL	AVAILABLE etc.
cancellation_reason	TEXT	NULLABLE	
cancelled_by_user_id	BIGINT	NULLABLE	
cancelled_at	TIMESTAMP	NULLABLE	
version	BIGINT	DEFAULT 0	@Version
created_at, updated_at	TIMESTAMP		
UNIQUE (doctor_id, slot_date, start_time)			

3 indexes + 1 unique constraint = 4 total
Scales to 100K+ rows with millisecond query times.

Three Critical Design Decisions

Decision 1: Denormalize current_bookings

Calculate from bookings table on every read would be slow for 1000-slot listings. Storing the count atomically as bookings are added/cancelled gives O(1) reads. Trade-off: small data duplication for huge query gains.

Decision 2: Store status explicitly

Could be calculated from current_bookings vs max_bookings, but that doesn't capture CANCELLED or EXPIRED. Explicit status field uniformly represents all four lifecycle states.

Decision 3: Per-slot max_bookings (not just config)

Storing max_bookings per slot allows: (a) historical accuracy when config changes, (b) per-slot overrides (special events, group consultations), and (c) one-row lookup without JOIN to config table.

Business Helper Methods

Helper methods on entity

```
public boolean isBookable() {
    return status == DoctorSlotStatus.AVAILABLE
        && currentBookings < maxBookings;
}

public Integer getRemainingSpots() {
    return Math.max(0, maxBookings - currentBookings);
}

public boolean isExpired() {
    LocalDateTime slotDateTime = LocalDateTime.of(slotDate, endTime);
    return slotDateTime.isBefore(LocalDateTime.now());
}
```

TIP » Business logic on the entity is fine for simple, self-contained checks. Anything involving other entities or repositories belongs in the service layer.

11. Indexing Strategy Deep Dive

Our 4 Indexes on doctor_slots

Index	Columns	Used By
PRIMARY	id	findById() - every entity lookup
uq_slot_doctor_date_time	doctor_id, slot_date, start_time	Duplicate prevention + checks
idx_slot_doctor_date	doctor_id, slot_date	MOST COMMON - 'Dr. X on date Y'
idx_slot_date_status	slot_date, status	Admin analytics, bulk expiry
idx_slot_status	status	Status-only filters, cleanups

Why Composite Index Order Matters (Left-Prefix Rule)

Left-prefix rule

Composite index: (doctor_id, slot_date)

QUERIES THAT USE IT:

WHERE doctor_id = 5	-> uses leftmost column
WHERE doctor_id = 5 AND slot_date = ?	-> uses BOTH (best!)
WHERE doctor_id = 5 AND slot_date > ?	-> uses range scan

QUERIES THAT DON'T USE IT:

WHERE slot_date = '2026-12-09'	-> NO! Skipped first column
WHERE slot_date BETWEEN ? AND ?	-> NO! Index unusable

That's why we have idx_slot_date_status (slot_date, status) too:
Different queries need different leading columns.

Cardinality and Index Usefulness

Cardinality = number of distinct values. High cardinality columns make great indexes. Status has cardinality of 4 (only 4 enum values) - by itself, indexing it doesn't help much because 'WHERE status = AVAILABLE' might still match 40% of rows. But combined with date (high cardinality), the composite (date, status) becomes very selective.

WARNING » Indexes make reads fast and writes slow. Each INSERT/UPDATE must update all relevant indexes. Don't index every column - profile your queries and index strategically.

12. Optimistic vs Pessimistic Locking

Both Are in Our Entity

DoctorSlot @Version field

```
// Optimistic locking - @Version field
@Version
@Column(name = "version", nullable = false)
private Long version = 0L;
```

Repository method with @Lock

```
// Pessimistic locking - in repository
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT s FROM DoctorSlot s WHERE s.id = :id")
Optional<DoctorSlot> findByIdWithLock(@Param("id") Long id);
```

Comparison Table

Aspect	Optimistic (@Version)	Pessimistic (@Lock)
Approach	Detect conflicts at save	Prevent conflicts upfront
DB lock?	No (lock-free)	Yes (row lock until commit)
Concurrency	High - no waits	Lower - threads wait
Best for	Conflicts rare	Conflicts likely
Performance	Best when uncontested	Best for hot rows
Failure mode	Throws on save (retry)	Waits for lock
When we use it	General entity updates	Critical booking path

The Race Condition @Version Prevents (Phase 4 Preview)

Race condition prevention

WITHOUT @Version:

```
T=0 Slot capacity = 2, current = 1 (one spot left)
T=10ms Patient A reads slot
T=11ms Patient B reads slot (both see 1 spot)
T=12ms A: current=2, save
T=13ms B: current=2, save
```

Result: Both saved! Slot oversubscribed!

WITH @Version (= 5 initially):

```
T=10ms A reads (version=5)
T=11ms B reads (version=5)
T=12ms A saves with WHERE version=5 -> success, version=6
T=13ms B saves with WHERE version=5 -> FAILS! 0 rows affected
```

JPA throws OptimisticLockException.
App catches and tells B "slot full, refresh".
No oversubscription!

INTERVIEW POINT » Interview Q: "How do you prevent double-booking?"

A: "Two-layer approach. Layer 1 is @Version (optimistic locking) — JPA auto-increments this on every update and uses it in WHERE clauses to detect concurrent modifications. Layer 2 is @Lock PESSIMISTIC_WRITE on a dedicated booking method — when a patient initiates booking, we acquire an exclusive DB row lock so no other transaction can read or update that slot until we commit. Pessimistic locking guarantees serialization for the critical booking section; optimistic locking is the safety net for general entity updates."

PART III · REPOSITORIES & QUERIES

13. The 4 Repositories

Repository	Methods	Special Features
DoctorSlotConfigRepository	3 simple	Just basic CRUD
DoctorWeeklyScheduleRepository	4 methods	Bulk delete by config
DoctorHolidayRepository	5 methods	Custom @Query for date ranges
DoctorSlotRepository	13+ methods	Specs, @Lock, @Modifying bulk ops

The Big One: DoctorSlotRepository

DoctorSlotRepository overview

@Repository

```
public interface DoctorSlotRepository extends JpaRepository<DoctorSlot, Long>,
                                             JpaSpecificationExecutor<DoctorSlot> {

    // Derived queries (Spring auto-generates SQL):
    List<DoctorSlot> findByDoctorAndSlotDateOrderByStartTimeAsc(...);
    boolean existsByDoctorAndSlotDateAndStartTime(...);
    long countByDoctorAndStatus(Doctor doctor, DoctorSlotStatus status);

    // Custom JPQL queries:
    @Query("SELECT s FROM DoctorSlot s WHERE s.status = 'AVAILABLE' ...")
    List<DoctorSlot> findAvailableSlotsByDoctorAndDate(...);

    // Pessimistic locking for booking:
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    Optional<DoctorSlot> findByIdWithLock(@Param("id") Long id);

    // Bulk UPDATE operations:
    @Modifying
    @Query("UPDATE DoctorSlot s SET s.status = 'CANCELLED' ...")
    int bulkCancelSlotsForDate(...);
}
```

14. Derived Queries, JPQL, and Specifications

Three Query Styles - When to Use Each

Style	Example	Best For
Derived (method name)	<code>findByDoctorAndSlotDate(d, date)</code>	Simple field-based queries
@Query (JPQL)	Custom SQL-like query	Complex conditions, joins
Specifications	Built at runtime via predicates	Optional/dynamic filters

Derived Query Examples

Spring magic

```
findByDoctorAndSlotDateOrderByStartTimeAsc(Doctor doctor, LocalDate date)
//      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//      Spring parses this and generates:
//      SELECT * FROM doctor_slots
//      WHERE doctor_id = ? AND slot_date = ?
//      ORDER BY start_time ASC
```

JPQL Example

Custom JPQL

```
@Query("SELECT s FROM DoctorSlot s " +
        "WHERE s.doctor.id = :doctorId " +
        "AND s.slotDate = :date " +
        "AND s.status = 'AVAILABLE' " +
        "ORDER BY s.startTime ASC")
List<DoctorSlot> findAvailableSlotsByDoctorAndDate(...);
```

JSQL looks like SQL but uses entity names. **DoctorSlot** is the Java class, not the table name. **s.doctor.id** navigates the relationship object, not the FK column. JPA translates this to actual SQL.

15. Bulk Operations with @Modifying

The Power of Bulk Updates

Holiday auto-cancel implementation

```
@Modifying
@Query("UPDATE DoctorSlot s SET " +
      "s.status = 'CANCELLED', " +
      "s.cancellationReason = :reason, " +
      "s.cancelledByUserId = :cancelledByUserId, " +
      "s.cancelledAt = :cancelledAt " +
      "WHERE s.doctor.id = :doctorId " +
      "AND s.slotDate = :date " +
      "AND s.status IN ('AVAILABLE', 'FULLY_BOOKED')")
int bulkCancelSlotsForDate(...);
```

Performance Comparison

Bulk vs individual

SCENARIO: Cancel 20 slots when admin adds a holiday

WITHOUT bulk (entity-by-entity):

```
SELECT * FROM doctor_slots WHERE doctor=5 AND date='2026-11-12' -- 1 query
for each slot:
  UPDATE doctor_slots SET status='CANCELLED' WHERE id=X          -- 20 updates
TOTAL: 21 queries, ~200ms
```

WITH bulk @Modifying:

```
UPDATE doctor_slots SET ... WHERE doctor=5 AND date='2026-11-12' -- 1 query
TOTAL: 1 query, ~5ms
```

40x faster!

COMMON MISTAKE » @Modifying queries bypass JPA's first-level cache. After running one, refresh affected entities in your transaction or the cache will hold stale data. Spring Boot's @Modifying(clearAutomatically = true) does this automatically.

16. Composite Index Left-Prefix Rule

The Rule in One Sentence

A composite index on (A, B, C) only helps queries that filter starting from column A. Skipping the leftmost column makes the index unusable.

Visual Example

Left-prefix visual

Composite index: `idx_slot_doctor_date (doctor_id, slot_date)`

WORKS:

```
WHERE doctor_id = 5
WHERE doctor_id = 5 AND slot_date = '2026-12-09'    <- BEST!
WHERE doctor_id = 5 AND slot_date BETWEEN ? AND ?
WHERE doctor_id = 5 ORDER BY slot_date               <- sorts use index
```

DOES NOT WORK:

```
WHERE slot_date = '2026-12-09'                      <- skips doctor_id
WHERE slot_date BETWEEN ? AND ?
ORDER BY slot_date (no WHERE)
```

Why? B-Tree indexes sort by FIRST column, then SECOND within each first-column value. You can't seek into the middle of a tree without knowing the first level.

How to Verify with EXPLAIN

Verify index usage

```
EXPLAIN SELECT * FROM doctor_slots
WHERE doctor_id = 5 AND slot_date = '2026-12-09';
```

```
-- Output:
-- key: idx_slot_doctor_date  <- index used!
-- rows: 20                  <- only scans 20 rows
```

INTERVIEW POINT » Interview Q: "You have an index on (col_A, col_B). Why doesn't your query filtering on col_B use it?"

A: "Left-prefix rule. Composite indexes work like a phone book sorted by last_name first, then first_name. You can find 'Smith, John' fast. But finding 'all Johns' requires reading the whole book. To support queries that lead with col_B, you need a separate index starting with col_B. I do this in our slot table: `idx_slot_doctor_date (doctor_id, slot_date)` for doctor-led queries, and `idx_slot_date_status (slot_date, status)` for date-led queries."

PART IV · DTO & MAPPING

17. 11 DTOs and Their Purposes

DTO	Type	Purpose
SlotConfigRequest	Request	Admin creates/updates slot config
WeeklyScheduleRequest	Request (nested)	Per-day schedule in config
GenerateSlotsRequest	Request	Trigger slot generation
HolidayRequest	Request	Add single holiday
BulkHolidayRequest	Request	Add multiple holidays
CancelSlotRequest	Request	Cancel slot with reason
SlotConfigResponse	Response	Full config with schedules
WeeklyScheduleResponse	Response (nested)	Per-day schedule data
HolidayResponse	Response	Holiday with audit
SlotResponse	Response	Full slot details (auth)
SlotSummaryResponse	Response	Minimal slot (public)

18. Nested DTOs with @Valid

The Critical @Valid Annotation

Top-level DTO

```
public class SlotConfigRequest {  
  
    @NotNull @Min(5) @Max(240)  
    private Integer slotDurationMinutes;  
  
    @Valid                                // <-- CRITICAL!  
    @Size(max = 7)  
    private List<WeeklyScheduleRequest> weeklySchedules;  
}
```

WARNING » Without **@Valid** on the nested list, validation annotations INSIDE `WeeklyScheduleRequest` are SILENTLY IGNORED. This is the #1 gotcha with nested validation. Always cascade **@Valid** through nested objects.

What Happens Without @Valid

@Valid cascade

WITHOUT **@Valid** on the nested list:

- Top-level **@NotNull**, **@Min** validate OK
- Each `WeeklyScheduleRequest`'s **@NotNull**, **@Min** are IGNORED
- Bad data flows through to service
- 500 errors instead of 400 Bad Request

WITH **@Valid** on the nested list:

- Full validation cascades down
- Each `WeeklyScheduleRequest` validated
- Bad data rejected at controller boundary
- Clean 400 with field-level error messages

19. Bidirectional Collection Management

The Pattern

Helper methods on parent entity

```
// In DoctorSlotConfig entity:
public void addWeeklySchedule(DoctorWeeklySchedule schedule) {
    weeklySchedules.add(schedule);    // Side A
    schedule.setSlotConfig(this);      // Side B - link back!
}

public void removeWeeklySchedule(DoctorWeeklySchedule schedule) {
    weeklySchedules.remove(schedule);
    schedule.setSlotConfig(null);      // Break the link
}
```

Why Helper Methods Matter

Why use helpers

WITHOUT helpers (manual, error-prone):
config.getWeeklySchedules().add(newSchedule);
newSchedule.setSlotConfig(config); // Easy to forget!

If forgotten: schedule is in parent's list but has null parent FK
-> Constraint violation on save
-> Inconsistent in-memory state

WITH helpers:
config.addWeeklySchedule(newSchedule); // Both sides linked atomically

Impossible to forget the back-link.

In the Mapper Update Logic

Update flow in mapper

```
public void updateEntity(DoctorSlotConfig config, SlotConfigRequest request) {
    config.setSlotDurationMinutes(request.getSlotDurationMinutes());
    // ... other fields

    // Clear existing (orphanRemoval=true triggers DELETE)
    config.getWeeklySchedules().clear();

    // Rebuild
    buildWeeklySchedules(config, request.getWeeklySchedules());
}

private void buildWeeklySchedules(DoctorSlotConfig config, ...) {
    for (DayOfWeek day : DayOfWeek.values()) {
        DoctorWeeklySchedule schedule = ...;
        config.addWeeklySchedule(schedule); // Use helper!
    }
}
```

TIP » The clear()-and-rebuild pattern works WITH JPA orphanRemoval. Setting a new list (setWeeklySchedules(newList)) BREAKS orphanRemoval. Always modify the existing list.

PART V · SLOT GENERATION ALGORITHM

20. The Algorithm Explained

In Plain English

Algorithm overview

INPUTS:

- Doctor (e.g., Dr. Smith)
- SlotConfig (30 min slots, max 2 bookings)
- WeeklySchedules (Mon-Fri 10-20, Sat 10-14, Sun closed)
- Holidays (Dec 25 etc.)
- Date range (e.g., Dec 1-31)

ALGORITHM:

1. Load all holidays for date range as a SET (one DB query)
2. Build a MAP from DayOfWeek to schedule (O(1) lookup)
3. For each date from startDate to endDate:
 - a. Is it a holiday? -> skip
 - b. Look up schedule by day of week
 - c. Is it a non-working day? -> skip
 - d. Do slots already exist? -> skip (if skipExisting=true)
 - e. Generate slots:
 - Loop work_start to work_end stepping by duration:
 - create slot(start, start+duration)
4. Bulk save all generated slots

OUTPUT:

List of DoctorSlot entities ready for persistence

Time Complexity

Operation	Complexity
Total iterations	$O(\text{days} \times \text{slots_per_day})$
Holiday check	$O(1)$ via HashSet
Schedule lookup	$O(1)$ via HashMap
Slot creation	$O(1)$ per slot
DB load (holidays)	$O(1)$ - single query
DB save (all slots)	$O(1)$ - single bulk insert

Total: **$O(\text{days} \times \text{slots_per_day})$** with minimal DB calls. For 30 days $\times$ 20 slots, that's 600 in-memory operations + 1 holiday query + 1 bulk insert. Sub-100ms execution.

21. Performance Optimizations (Set, Map)

Optimization 1: Prefetch Holidays as Set

Optimization in action

```
// BAD: N database queries
for each day in range:
    if (holidayRepository.existsByDoctorAndHolidayDate(doctor, day)) {
        // 30 DB calls for 30 days!
    }

// GOOD: 1 database query + O(1) lookups
Set<LocalDate> holidayDates = loadHolidayDates(doctor, start, end);
for each day in range:
    if (holidayDates.contains(day)) { // O(1) hash lookup
        // 0 additional DB calls
    }
```

Optimization 2: Map for Schedule Lookup

HashMap lookup

```
// BAD: Linear scan for every day in range
for each day in range:
    for each schedule in weeklySchedules:
        if (schedule.getDayOfWeek() == day.getDayOfWeek()) { ... }
// 30 days × 7 schedules = 210 comparisons

// GOOD: O(1) hashmap lookup
Map<DayOfWeek, DoctorWeeklySchedule> scheduleByDay = mapSchedulesByDay(config);
for each day in range:
    DoctorWeeklySchedule schedule = scheduleByDay.get(day.getDayOfWeek());
// 30 days × 1 lookup = 30 ops
```

Optimization 3: Bulk Save

Bulk save

```
// BAD: Individual saves
for (DoctorSlot slot : generatedSlots) {
    slotRepository.save(slot); // 88 round trips!
}

// GOOD: Bulk save
slotRepository.saveAll(generatedSlots); // Batched INSERTs
```

Why this matters » These three optimizations transform a naive $O(N^2)$ algorithm with $N+1$ DB queries into a clean $O(N)$ algorithm with just 2 DB queries. For a 30-day generation, the difference is approximately 5x faster execution.

22. Date Iteration with LocalDate

Java's java.time API Makes This Easy

Date range iteration

```
LocalDate currentDate = startDate;
while (!currentDate.isAfter(endDate)) {
    // process currentDate
    currentDate = currentDate.plusDays(1);
}
```

Time Arithmetic with LocalTime

Slot time loop

```
LocalTime currentStart = LocalTime.of(10, 0); // 10:00
int duration = 30;

while (true) {
    LocalTime currentEnd = currentStart.plusMinutes(duration);

    if (currentEnd.isAfter(endOfDay)) break; // Stop if past closing

    // create slot(currentStart, currentEnd)
    currentStart = currentEnd;
}
```

COMMON MISTAKE » WATCH OUT: LocalTime wraps around midnight! 23:30 + 60 min = 00:30. Our isAfter check prevents this for clinics that close before midnight. For 24/7 services or night shifts, use LocalDateTime to track the date too.

23. Worked Example: 88 Slots in 7 Days

Tracing the algorithm

INPUT:

```
date range: Dec 1-7, 2026
schedule:
  MON-FRI: 10:00-20:00 (full day)
  SAT:     10:00-14:00 (half day)
  SUN:     closed
holidays: { 2026-12-04 (Eid) }
slot duration: 30 minutes
max bookings: 2
```

EXECUTION DAY-BY-DAY:

Dec 1 (TUE):

```
holiday? NO
schedule? TUE = working, 10:00-20:00
Generate slots:
  10:00-10:30, 10:30-11:00, 11:00-11:30, ..., 19:30-20:00
= 20 slots
```

Dec 2 (WED): same as TUE = 20 slots

Dec 3 (THU): same = 20 slots

Dec 4 (FRI):

```
holiday? YES (Eid)
return empty = 0 slots
```

Dec 5 (SAT):

```
holiday? NO
schedule? SAT = working, 10:00-14:00
Generate slots:
  10:00-10:30, 10:30-11:00, ..., 13:30-14:00
= 8 slots
```

Dec 6 (SUN):

```
holiday? NO
schedule? SUN = non-working
return empty = 0 slots
```

Dec 7 (MON):

```
same as TUE-THU = 20 slots
```

TOTAL: $20+20+20+0+8+0+20 = 88$ slots

TIP » Always trace your algorithms by hand on a small example. If 88 slots is what you expect, and the algorithm produces 88, you have confidence in correctness. This is why testing matters - we verified this exact behavior in Postman.

PART VI · SERVICE LAYER

24. DoctorSlotConfigService

Responsibilities

- Create/update doctor's slot configuration
- Validate doctor is APPROVED before config setup
- Coordinate with SlotGenerationService for slot creation
- Activate/deactivate configurations

Service-to-Service Composition

Service composition

```
@Service
public class DoctorSlotConfigServiceImpl {

    private final SlotGenerationService slotGenerationService;
    // ... other dependencies

    @Transactional
    public int generateSlots(Long doctorId, GenerateSlotsRequest request) {
        // Validate doctor + config
        Doctor doctor = findDoctorById(doctorId);
        DoctorSlotConfig config = getConfigEntityForDoctor(doctor);

        if (doctor.getStatus() != DoctorStatus.APPROVED) {
            throw new BadRequestException("Doctor not APPROVED");
        }
        if (!config.isActive()) {
            throw new BadRequestException("Config not active");
        }

        // Delegate to generation service
        List<DoctorSlot> slots = slotGenerationService.generateSlots(
            doctor, config, request.getStartDate(),
            request.getEndDate(), request.getSkipExisting());

        // Bulk save
        slotRepository.saveAll(slots);
        return slots.size();
    }
}
```

Why this matters » Composition over inheritance. The config service ORCHESTRATES — validates inputs, calls the specialized generation service, persists results. The generation service stays focused on the algorithm. Clean separation of concerns.

25. DoctorHolidayService with Auto-Cancel

Your Design Choice in Action

You chose 'Both - holiday entity + auto-cancel slots' as the holiday handling approach. Here's how it works in the service:

Auto-cancel implementation

```
@Override
@Transactional
public HolidayResponse addHoliday(Long doctorId, HolidayRequest request, Long adminUserId) {
    Doctor doctor = findDoctorById(doctorId);

    // Validate no duplicate
    if (holidayRepository.existsByDoctorAndHolidayDate(doctor, request.getHolidayDate())) {
        throw new ResourceAlreadyExistsException(...);
    }

    // Create holiday entity
    DoctorHoliday holiday = holidayMapper.toEntity(request, doctor, adminUserId);
    holiday = holidayRepository.save(holiday);

    // Auto-cancel slots (the magic!)
    if (Boolean.TRUE.equals(request.getCancelExistingSlots())) {
        int cancelled = slotRepository.bulkCancelSlotsForDate(
            doctor.getId(),
            request.getHolidayDate(),
            "Holiday: " + request.getReason(),
            adminUserId,
            LocalDateTime.now());
        log.info("Auto-cancelled {} slots for {}", cancelled, request.getHolidayDate());
    }

    return holidayMapper.toResponse(holiday);
}
```

Why a Single Transaction?

Both operations (insert holiday + cancel slots) happen in ONE transaction. If either fails, everything rolls back. This guarantees consistency: you never end up with a holiday but non-cancelled slots, or cancelled slots without a holiday record.

Null-Safe Boolean Check

Null-safe pattern

```
// SAFE:
if (Boolean.TRUE.equals(request.getCancelExistingSlots())) { ... }

// UNSAFE (NPE if Boolean is null):
if (request.getCancelExistingSlots()) { ... }
```

TIP » Always use `Boolean.TRUE.equals()` when checking boxed Boolean values that might be null. This is a defensive programming standard for Java production code.

26. DoctorSlotService

Audience-Aware Methods

Audience-aware responses

```
AUTHENTICATED (Doctor/Admin):
  getSlotsForDoctorByDate(...) -> List<SlotResponse>      (FULL info)
  cancelSlot(...)              -> SlotResponse
  getSlotById(...)             -> SlotResponse

PUBLIC (Patient browsing):
  getAvailableSlotsByDoctorAndDate(...) -> List<SlotSummaryResponse> (MINIMAL)
  getNextAvailableSlots(...)           -> List<SlotSummaryResponse>
  getDatesWithAvailability(...)         -> List<LocalDate>
```

The service ITSELF enforces what's visible.
Even if a bug exposes a public endpoint to authenticated users,
sensitive data is filtered at the service layer.

Defense in depth.

DOS Protection in Public Methods

Bounded queries

```
@Override
@Transactional(readOnly = true)
public List<SlotSummaryResponse> getAvailableSlotsInRange(
    Long doctorId, LocalDate startDate, LocalDate endDate) {

    findDoctorById(doctorId); // Verify doctor exists

    if (startDate.isAfter(endDate)) {
        throw new BadRequestException("Start date must be before end date");
    }

    // DOS PROTECTION: cap range at 90 days
    long days = endDate.toEpochDay() - startDate.toEpochDay() + 1;
    if (days > 90) {
        throw new BadRequestException("Range cannot exceed 90 days");
    }

    // ... proceed with query
}
```

WARNING » Public endpoints are reachable without authentication. ALWAYS cap inputs (date ranges, result limits, page sizes). A malicious user can craft requests that cause expensive queries — the cap is your line of defense.

27. Defense in Depth Validation

Five Layers of Validation

Validation layers

LAYER 1: HTTP Method + Path

Spring Security: only authorized roles/methods reach the controller

LAYER 2: Bean Validation on DTOs

@NotNull, @Min, @Max, @Size, @Pattern

Caught at controller boundary (400 Bad Request)

LAYER 3: Custom validation in DTOs

Cross-field rules (start_time < end_time)

Caught at service entry

LAYER 4: Service-level business rules

Doctor must be APPROVED

Config must be active

State machine transitions valid

LAYER 5: Database constraints

UNIQUE constraints

NOT NULL constraints

Foreign keys

Each layer catches DIFFERENT bugs.

A typo bypasses layer 2 (gets to service),

but layer 4 catches a missing entity.

A buggy client bypasses layer 1,

but layer 5 catches a duplicate key.

DEFENSE IN DEPTH = redundant safety nets.

INTERVIEW POINT » Interview Q: "How do you ensure data integrity in your application?"

A: "Defense in depth across five layers. Spring Security at the perimeter, Bean Validation at the controller, custom validation rules, business invariants in the service layer, and finally database constraints as the last line of defense. Each layer catches different categories of bugs. Relying on only one layer (say, just Bean Validation) leaves gaps - a service can be called by another service that bypasses Bean Validation entirely. Service-level business rules are non-negotiable."

PART VII · CONTROLLERS & SECURITY

28. Three Controllers Pattern

Same Pattern as Phase 2

Three controllers

```
+-----+
| DoctorSlotController          |
| @PreAuthorize("hasRole('DOCTOR')") |
| /doctors/me/slots            |
+-----+

+-----+
| AdminSlotController           |
| @PreAuthorize("hasRole('ADMIN')") |
| /admin/...                    |
+-----+

+-----+
| PublicSlotController          |
| (no @PreAuthorize - public access) |
| /doctors/{id}/slots/available    |
+-----+
```

Each controller has ONE audience.

Each uses ONE response DTO style.

Security is class-level, no per-method overrides.

Why this matters » Splitting by audience makes URL paths intuitive. **/doctors/me/...** is obviously self-service. **/admin/...** is obviously admin-only. **/doctors/{id}/slots/available** is obviously public. Recruiters or new team members can understand the API surface at a glance.

29. @DateTimeFormat for Date Parsing

Date parsing

@GetMapping

```
public ResponseEntity<...> getMySlots(  
    @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate startDate,  
    @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate endDate,  
    ...) {
```

Why This Annotation?

DateTimeFormat magic

WITHOUT @DateTimeFormat:

```
GET /doctors/me/slots?startDate=2026-12-09
```

Spring tries to convert "2026-12-09" to LocalDate

-> No converter registered for String -> LocalDate from query param

-> 500 error or fallback parsing failure

WITH @DateTimeFormat(iso = DateTimeFormat.ISO.DATE):

Spring knows to expect ISO 8601 format (YYYY-MM-DD)

-> Clean conversion

-> Validation errors return 400 with field info

TIP » ISO 8601 (YYYY-MM-DD) is the global standard. Use this format for ALL date APIs unless you have a specific reason to use a localized format. Eliminates timezone and locale issues.

30. Regex in URL Patterns (Security!)

The Path Conflict Problem

Why regex matters

We have these endpoints:

```
/doctors/me/slots      -> requires DOCTOR auth  
/doctors/5/slots/available -> public access
```

In SecurityConfig, we want:

```
.requestMatchers(HttpMethod.GET, "/doctors/{id}/slots/available").permitAll()
```

WITHOUT regex constraint:

```
/doctors/me/slots/available -> matches {id}=me!  
-> public access granted to /doctors/me path  
-> SECURITY BREACH (user identification info exposed)
```

WITH regex {id:[0-9]+}:

```
.requestMatchers(GET, "/doctors/{id:[0-9]+}/slots/available").permitAll()
```

```
/doctors/5/slots/available -> matches (numeric)  
/doctors/me/slots/available -> DOES NOT MATCH  
-> falls through to .anyRequest().authenticated()  
-> SAFE
```

COMMON MISTAKE » Forgetting regex on numeric IDs is a SUBTLE SECURITY BUG. The code 'works' but exposes private endpoints. Always use regex constraints when public and private paths share a URL prefix. Test this explicitly in security tests.

31. SecurityConfig Updates

The 4 Lines We Added

Updated rules

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/auth/**").permitAll()
    .requestMatchers("/actuator/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/search").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/specializations").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/{id:[0-9]+}").permitAll()

    // === Phase 3 additions ===
    .requestMatchers(HttpMethod.GET, "/doctors/{id:[0-9]+}/slots/available").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/{id:[0-9]+}/slots/available/range").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/{id:[0-9]+}/slots/available/next").permitAll()
    .requestMatchers(HttpMethod.GET, "/doctors/{id:[0-9]+}/slots/dates").permitAll()

    .anyRequest().authenticated()
)
```

Three Critical Rules

- **HttpMethod.GET** only — never accidentally expose POST/PUT/DELETE
- **{id:[0-9]+}** regex — only numeric IDs match
- **Most-specific first** — catch-all `.anyRequest()` comes last

PART VIII · REFERENCE

32. All Endpoints Cheat Sheet

Method	Endpoint	Role
POST	/doctors/me/slots/{id}/cancel	DOCTOR
GET	/doctors/me/slots	DOCTOR
GET	/doctors/me/slots/{id}	DOCTOR
POST	/admin/doctors/{id}/slot-config	ADMIN
GET	/admin/doctors/{id}/slot-config	ADMIN
PUT	/admin/doctors/{id}/slot-config/active	ADMIN
POST	/admin/doctors/{id}/slots/generate	ADMIN
POST	/admin/doctors/{id}/holidays	ADMIN
POST	/admin/doctors/{id}/holidays/bulk	ADMIN
GET	/admin/doctors/{id}/holidays	ADMIN
GET	/admin/doctors/{id}/holidays/upcoming	ADMIN
DELETE	/admin/holidays/{id}	ADMIN
GET	/admin/doctors/{id}/slots	ADMIN
POST	/admin/slots/{id}/cancel	ADMIN
GET	/doctors/{id}/slots/available	Public
GET	/doctors/{id}/slots/available/range	Public
GET	/doctors/{id}/slots/available/next	Public
GET	/doctors/{id}/slots/dates	Public

33. Postman Test Suite Recap

Smoke Tests Validated

Area	What Was Verified
Authentication	Admin + Doctor JWTs obtained
Slot Config	Create with 7-day per-day schedule
DB Verification	7 weekly_schedule rows, 1 config row
Slot Generation	Correct slot counts per day (20/8/0)
Public Browsing	Only AVAILABLE slots, minimal data
Sunday/Holiday	Empty list returned
Holiday Magic	20 slots auto-cancelled on holiday add
Bulk Holidays	Multiple dates inserted in one transaction
Doctor Cancel	Single slot status changed to CANCELLED
Hidden from Public	Cancelled slots not in public list
Role Security	DOCTOR token can't hit /admin endpoints
No-Auth Security	/me path blocked without JWT
Regex Security	/doctors/me/slots/... not public

TIP » All tests passed in one session. This proves the entire Phase 3 system works end-to-end. The slot generation algorithm, holiday auto-cancel, role-based access, and public/private data filtering all integrate correctly.

34. Common Mistakes & Fixes

Mistake	Symptom	Fix
No @Valid on nested DTO list	Inner validations ignored	Add @Valid
Setting list (vs clearing)	orphanRemoval doesn't work	list.clear(); list.add(...)
EnumType.ORDINAL	Data corruption on reorder	Always EnumType.STRING
LocalDateTime for holidays	Time component is irrelevant	Use LocalDate
Forgetting @Modifying	Update query fails	Add @Modifying for UPDATES
No regex on numeric IDs	/me path becomes public	Use {id:[0-9]+}
Missing HttpMethod.GET	All methods become public	Specify method
No DOS cap on public ranges	Huge query DOS attack	Cap at 90 days
Calling save() in @Transactional	Redundant, can cause issues	Use dirty checking
Boolean unboxing NPE	NullPointerException	Boolean.TRUE.equals(...)
Individual saves in loop	100x slower than bulk	saveAll() or @Modifying bulk
Restoring slots on holiday delete	Confusing side effect	Keep cancellations explicit

35. Interview Q&A; for Phase 3

O1: Walk me through your slot management design.

INTERVIEW POINT » "Four entities for normalization. DoctorSlotConfig holds universal settings (slot duration, max bookings, timezone). DoctorWeeklySchedule stores per-day working hours (7 rows per doctor). DoctorHoliday tracks exception dates. DoctorSlot is the actual bookable time window with current_bookings denormalized for performance. A SlotGenerationService takes the config and schedule, iterates date ranges, skips holidays via O(1) HashSet lookups, and produces slots for bulk insert. Optimistic locking via @Version prevents double-booking. All four entities are wired through clean service interfaces with state machine validation in the service layer."

O2: How does your slot generation algorithm work?

INTERVIEW POINT » "It's a date-range iterator with three optimizations. First, I pre-load all holidays for the range into a HashSet for O(1) membership checks - instead of querying per day. Second, I build a HashMap from DayOfWeek to schedule for O(1) day-of-week lookups - instead of scanning the schedule list each iteration. Third, I use saveAll() to bulk-insert the generated slots in a single batch. The algorithm validates inputs upfront (max 365 days), then for each date: checks if holiday, looks up the day's working hours, and steps from work_start to work_end by slot_duration_minutes. Result is O(n) where n = days × slots_per_day."

O3: How do you handle the holiday auto-cancel feature?

INTERVIEW POINT » "When admin adds a holiday and existing slots already exist for that date, I use a @Modifying bulk UPDATE query - one SQL statement that cancels all matching slots in a single round trip. The whole operation runs in one @Transactional method: insert holiday + bulk cancel slots. If either fails, both roll back. The bulk update sets status=CANCELLED, the cancellation reason 'Holiday: X', who cancelled, and timestamp. For 20 affected slots, this is one query vs 20 individual updates - about 40x faster."

O4: How do you prevent oversubscription / double-booking?

INTERVIEW POINT » "Two layers. The DoctorSlot entity has @Version (JPA optimistic locking). When a booking transaction reads a slot, the version is captured. On save, JPA appends WHERE version = X. If another transaction already updated the row (incrementing version), our save affects 0 rows and throws OptimisticLockException. For Phase 4's critical booking path, I also have a @Lock PESSIMISTIC_WRITE repository method - it acquires a DB row lock at SELECT time, so no other transaction can even READ that slot until we commit. Pessimistic locking serializes the critical section; optimistic locking is the general-purpose safety net."

O5: How do you design indexes for performance?

INTERVIEW POINT » "I follow query patterns, not guesses. For doctor_slots, the most common query is 'show me Dr. X's slots on date Y' - that drives idx_slot_doctor_date (doctor_id, slot_date). Admin analytics often filter by (slot_date, status) for queries like 'available slots today' - that's idx_slot_date_status. Composite index order matters because of the left-prefix rule - WHERE clauses must filter starting from the leftmost column. I also added a unique constraint on (doctor_id, slot_date, start_time) for data integrity. Trade-off: each index slows down INSERTs about 10-15%, but slots are write-once-read-many so it's worth

it."

O6: How would you scale this to millions of slots?

INTERVIEW POINT » "Several techniques. First, partition the slots table by month - queries usually target recent dates, so old partitions can be archived. Second, add a Redis cache for the most-read queries (today's available slots per doctor). Third, move slot generation to an async job (Phase 5) - admin triggers it, background worker processes. Fourth, add a slot expiry job that runs nightly to mark past slots EXPIRED in bulk. Fifth, for very high concurrency, consider an in-memory inventory system (Redis-based) backed by the database. For Phase 3's scale (~100K slots), the current schema with proper indexes handles everything in milliseconds."

36. Phase 4 Preview: Booking System

What's Coming Next

Phase 4 will add the booking layer on top of slots. Patients will be able to actually reserve slots. This is where Phase 3's @Version and @Lock investments pay off.

New Entities

- **Booking entity** — links Patient + Slot + status (BOOKED, COMPLETED, CANCELLED, NO_SHOW)
- **BookingHistory** — audit trail of booking changes

New Concepts You'll Learn

- Concurrency control with @Lock PESSIMISTIC_WRITE (the real use case!)
- Distributed transactions
- Retry logic on OptimisticLockException
- Cancellation refund policies
- No-show tracking
- Booking conflict resolution

Interview Gold

Phase 4 will give you concrete answers to interview questions about: race conditions, consistency vs availability, idempotent operations, payment integration patterns, and microservice boundaries.

PHASE 3 COMPLETE!

You now have a production-grade slot management system with:
Per-day schedules · Holiday auto-cancel · Slot generation
Optimistic + pessimistic locking · Defense in depth

Repository: github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook

3 of 11 phases complete — ready for Phase 4!